

Enterprise Release Notes Automation Pipeline

Gila (Geetha) Nehemia · gnehemia@gmail.com · linkedin.com/in/gnehemia

PowerFlex release notes are how a customer finds out what was fixed, what was deferred, and what is new. When they are wrong or inconsistent, the customer pays for it in support tickets, failed upgrades, and time spent guessing. Producing them meant extracting issue data out of Jira by hand, then reviewing every defect and fix several times over, once for technical accuracy, again for adherence to the company style guide, again after the data changed. Every manual pass was a chance for a detail to drift, and the details that drift are exactly the ones customers depend on.

I decided to build this out of deep frustration with that process. A colleague and I had named the problem a full year before any automation existed, and we did not jump to a tool. We documented the pain points, wrote design documentation, a phased pilot plan, and a testing plan, and only then started building. The insight that made it work was that my own release note workflow, built over years of enterprise documentation practice, could be broken into discrete, testable steps and encoded. Instead of teaching a model to write release notes from scratch, I digitized expertise I already had and let it run as a guardrail at every step.

The pipeline is six modular skills, each consuming the output of the one before it. One queries Jira for deferred issues and drafts them. Another scans that system and the enterprise knowledge base for resolutions and annotates the master file, so it always reflects current release state. A transformation engine rebuilds the known issues file and generates two more from embedded XML templates. A fourth pulls new customer-facing capabilities from Jira and the design platform at once, validating the data before it reaches content generation and reusing shared skills built by other teams. A shared content generation engine, called by two of the skills, applies the institutional style guide and terminology file as structural guardrails rather than after-the-fact checks. The last posts a comment to the responsible subject matter expert when a new deferred issue is documented, so review still happens where the expertise lives and the automation does the tracking. One run produces five compliant, correctly structured outputs, already routed for review.

It was engineered in the order a professional AI team would use. The deterministic, rule-based version came first, so the foundation was predictable before any intelligence sat on top of it. JSON logging made every action traceable. Python moved structured data between enterprise systems at each stage. A Devin-native orchestrator now sequences three of the core skills in a single invocation, using MCP server integration to reach Jira and the file system directly and reading each skill's logged output to route the next one. It is an orchestrator I built in Devin, and it automates whole sections of the PowerFlex release notes end to end. During testing, the model flagged a version mismatch unprompted and asked whether to extend the logic to the next release. Direct API access to the enterprise CCMS is in progress and will remove the final manual step.

Building each skill as a small, discrete unit means a failure can be isolated at the module level without disrupting the rest of the pipeline. Build once, reuse everywhere.

A pipeline only one person can run is a bottleneck, not a solution. Alongside the build I wrote the design documentation and the user guides, then trained a colleague until they could run and maintain the pipeline independently. That transfer is what let the next piece be built collaboratively rather than by me alone. The effort has since moved past a single team. I built a template capturing what works and the time it saves, and gave it to team leads across the organization so each team can document its own process against a common structure.

The most recent extension adapts the pipeline from NPI releases to in-market service pack releases, the smaller, more frequent updates issued after launch. Rather than build a parallel system, it reuses the same skills and content generation engine at that cadence. My colleague and I made the adjustments needed to meet service pack guidelines in two to three days, working a few hours a day.

Both pipelines are past pilot and in pre-production, with formal metrics collected. Working the process through with Devin, and the XML automation that followed, has saved us more than 70% of the manual effort of extracting the data and editing the content customers read. That number matters mostly for what it bought. Data now comes straight from Jira instead of being re-keyed, customers see the same structure and terminology every cycle, cross-referencing catches what a manual pass misses, and the time comes back to the judgment work automation cannot do, which is deciding what a change actually means for a customer and saying it clearly.

Python · Jira (JQL) · JSON processing · XML manipulation · Devin · MCP server integration · orchestration · agentic workflow design · modular pipeline architecture · enterprise CCMS